SOFTWARE ENGINEERING · CHAPTER 12

Lab-12: Deployment Diagrams

You take the ITC Campus Library System modelled in Labs 07-11 and plan its deployment for the whole university (about 5 000 members, peak load at semester start). Across five tasks you draw and build: a single-server deployment, the artifacts and their deployment specifications, a production topology with zones and firewalls, a Kubernetes deployment, and the dev/test/prod variants with security and monitoring. The challenge adds a blue/green release design, a CI/CD pipeline, a capacity estimate and a disaster-recovery plan.

UML 2.5.1 · PlantUML JDK 25 · Spring Boot 4.1 · Docker · Kubernetes ≈ 6 hours 5 tasks + 1 challenge

How to work through this lab. Do the tasks in order; each builds on the previous. Read the *Goal*, follow the *Steps*, compare with the *Expected output*, and tick the *Acceptance checklist* (saved in your browser). Open a hint only when stuck for more than 10 minutes. Diagrams may be drawn in PlantUML, draw.io or Mermaid-free SVG, but you must submit the **source text** and a **PNG or SVG export** of every diagram.

0 Setup — 20 min

1. Verify the tool chain. Docker Desktop (or Podman) with Compose v2 and a local Kubernetes (Docker Desktop's built-in cluster, kind or minikube) are required for Tasks 2-4.

```
java -version            # openjdk 25
mvn -version             # Apache Maven 3.9.16
docker --version         # Docker 27+
docker compose version   # v2.x
kubectl version --client # v1.31+
kind version             # or: minikube version
```

2. Install PlantUML: `plantuml.jar` (needs Java and Graphviz) or the VS Code / IntelliJ PlantUML plugin. Check that `node`, `artifact`, `database`, `queue` and `cloud` render:

```
java -jar plantuml.jar -tpng diagrams/*.puml
```

3. Reuse the library project from Labs 07-11 (`edu.itc.library`, Spring Boot 4.1.1, Spring Data JPA, Spring Security, Actuator). If you do not have it, generate a minimal one with `spring-boot-starter-webmvc`, `spring-boot-starter-data-jpa`, `spring-boot-starter-`

`actuator`, `spring-boot-starter-security`, `postgresql` and `h2`, and the component names from Lab-11: `Catalog`, `Loans`, `Members`, `Notifications`, `Persistence`.

4. Create the folder layout used by every task:

```
library-deploy/
├── README.md
├── diagrams/            # *.puml sources + *.png exports
├── app/                 # the Spring Boot project (pom.xml, src/)
├── docker/              # Dockerfile, docker-compose.yml, nginx/,
prometheus/
├── k8s/                 # base/ and overlays/{dev,test,prod}/
└── runbooks/            # markdown runbooks (Task 5, challenge)
```

5. Add `management.endpoint.health.probes.enabled=true` and expose `health,info,prometheus` in `application.yml`; add `io.micrometer:micrometer-registry-prometheus`. Confirm `curl localhost:8080/actuator/health/readiness` returns `{"status":"UP"}` with the app running.

1 First deployment diagram: one server

Easy **40 min**

GOAL

Draw the simplest correct deployment of the library system: browsers on member devices, one virtual machine running the JVM and the Spring Boot jar, PostgreSQL on the same VM. Use the exact UML 2.5.1 notation: 3D nodes with stereotypes, nested execution environments, an artifact with the document icon, communication paths labelled with protocol and port.

STEPS

1. Create `diagrams/01-single-server.puml`. Add a `«device»` node `Member laptop / phone [*]` containing an execution environment `Browser`.
2. Add the `«device»` node `lib-vm-01` (annotate: Ubuntu 24.04, 2 vCPU, 4 GB). Inside it nest `«executionEnvironment» JVM 25`, and inside that `«executionEnvironment» Embedded Tomcat 11` holding the artifact `library-1.0.jar`.
3. Inside the same VM add a second execution environment `PostgreSQL 17` with the artifact `V1__schema.sql` deployed on it (Flyway migration).
4. Draw two communication paths: browser → VM labelled `HTTPS :443`, and JVM → PostgreSQL labelled `JDBC :5432 (localhost)`. Decide and note who opens each connection.

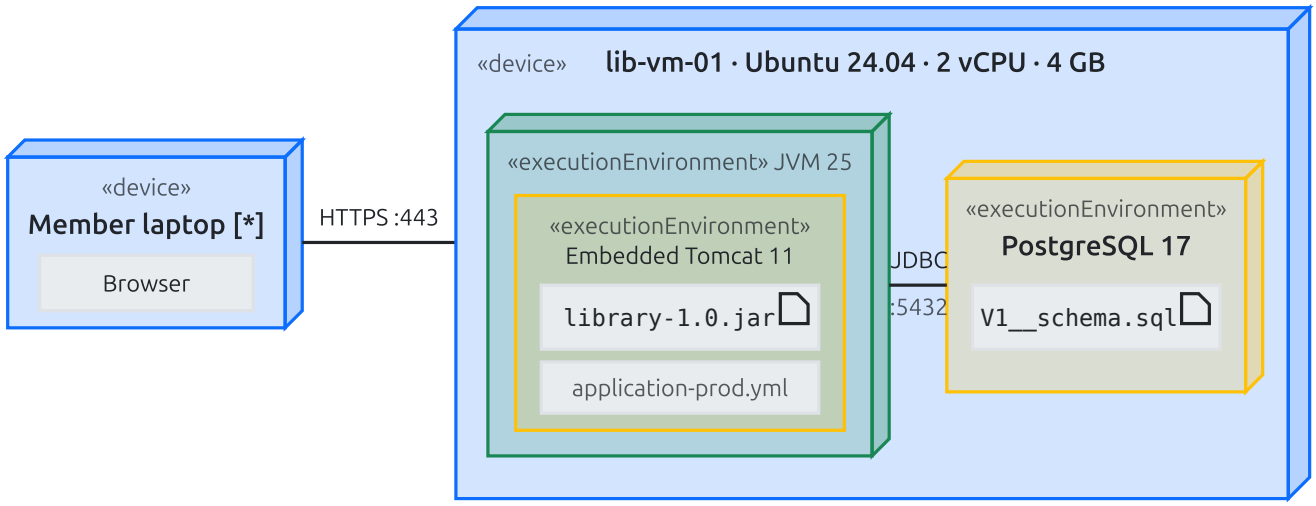
5. Add a `«deployment spec»` artifact `application-prod.yml` next to the jar with a note listing `server.port`, `spring.datasource.url` and `SPRING_PROFILES_ACTIVE=prod`.
6. Export to `diagrams/01-single-server.png`. In `README.md` write three sentences: what is a device here, what is an execution environment, and why the DB on the same VM is a single point of failure.
7. Verify the diagram against the ten-point checklist from slide 24 and list which points cannot be satisfied yet (zones, multiplicity on servers, monitoring) so Tasks 3-5 can close them.

STARTER DIAGRAM TEXT

```
@startuml 01-single-server
skinparam nodesep 40
' TODO: multiplicity [*] on the client device
node "Member laptop / phone" <<device>> as client {
  node "Browser" <<executionEnvironment>>
}
node "lib-vm-01\nUbuntu 24.04 · 2 vCPU · 4 GB" <<device>> as vm {
  node "JVM 25" <<executionEnvironment>> as jvm {
    node "Embedded Tomcat 11" <<executionEnvironment>> {
      artifact "library-1.0.jar" as jar
      ' TODO: artifact "application-prod.yml" <<deployment spec>>
    }
  }
  ' TODO: node "PostgreSQL 17" <<executionEnvironment>> { artifact
"V1__schema.sql" }
}
client -- vm : HTTPS :443
' TODO: jvm -- postgres : JDBC :5432 (localhost)
@enduml
```

EXPECTED OUTPUT

The exported PNG shows the structure below (rendered here as SVG): one client device with multiplicity, one VM with two nested execution environments, the jar and the SQL script as artifacts, two labelled paths.



Show hints

- PlantUML draws a 3D box for `node`; the stereotype goes in `<< >>` after the name. Multiplicity is just part of the name: `node "Member laptop [*]"`.
- Nesting = curly braces. Everything inside the braces is drawn inside the parent node.
- A communication path is `a -- b : label`; use `a --> b` only if you want to show who initiates the connection.
- Use `database "PostgreSQL 17"` if you prefer the cylinder; UML-wise it is still an execution environment on the VM, say so in the README.

ACCEPTANCE CHECKLIST

- [ ] Every node carries «device» or «executionEnvironment»; the jar and the SQL script are artifacts inside the environment that executes them.
- [ ] Both communication paths are labelled with protocol and port; the client device has multiplicity `[*]`.
- [ ] The deployment specification is drawn and its three parameters are listed.
- [ ] `.puml` source and PNG are committed; README explains the single point of failure and lists the open checklist points.

2 Artifacts and deployment specifications

Easy 50 min

GOAL

Produce the real artifacts: the executable jar and a Docker image built with a multi-stage Dockerfile. Document the deployment specification as `application-prod.yml` plus

environment variables, and draw the manifestation diagram that connects the Lab-11 components to these artifacts and to the nodes.

STEPS

1. Build the jar: `./mvnw -q -DskipTests package` in `app/`. Record the file name and size in the README; run `java -jar target/library-1.0.jar --spring.profiles.active=dev` and confirm the Actuator health endpoint answers.
2. Write `docker/Dockerfile`: stage 1 `eclipse-temurin:25-jdk` runs the Maven build (cache `dependency:go-offline` before copying `src/`); stage 2 `eclipse-temurin:25-jre` copies only the jar, creates user `app` (uid 1001), sets `USER 1001`, `EXPOSE 8080`, `JAVA_TOOL_OPTIONS=-XX:MaxRAMPercentage=75` and a `HEALTHCHECK` on `/actuator/health`.
3. Build and run: `docker build -f docker/Dockerfile -t itc/library:1.0 app/`, then `docker run --rm -p 8080:8080 -e SPRING_PROFILES_ACTIVE=dev itc/library:1.0`. Print the image digest with `docker inspect --format '{{index .RepoDigests 0}}' itc/library:1.0` (or `docker images --digests`).
4. Create `app/src/main/resources/application-prod.yml`: datasource from `${DB_URL}`, `${DB_USER}`, `${DB_PASSWORD}`; `server.tomcat.threads.max: 100`; Hikari `maximum-pool-size: 10`; `spring.jpa.hibernate.ddl-auto: validate`; Flyway enabled; Actuator probes enabled. No secret value may appear in the file.
5. Write `docker/.env.example` with every environment variable the prod profile needs (names only, placeholder values) and a table in the README: variable → purpose → where it is stored in prod.
6. Draw `diagrams/02-artifacts.puml`: the five Lab-11 components (`Catalog`, `Loans`, `Members`, `Notifications`, `Persistence`) with `«manifest»` arrows from the artifacts `library-1.0.jar` and `itc/library:1.0` (image contains the jar), `V*.sql` manifesting `Persistence`, and `«deploy»` arrows to the nodes `lib-vm-01` and `PostgreSQL 17`. Attach the deployment specification to the jar.
7. Export the PNG and answer in the README: which artifact is the unit of promotion between environments (jar or image), and why the digest matters more than the tag.

STARTER CODE

```
# docker/Dockerfile
FROM eclipse-temurin:25-jdk AS build
WORKDIR /src
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN ./mvnw -q dependency:go-offline
COPY src/ src/
RUN ./mvnw -q -DskipTests package
```

```
FROM eclipse-temurin:25-jre
# TODO: create user app (uid 1001), WORKDIR /opt/library
COPY --from=build /src/target/library-1.0.jar library.jar
# TODO: USER 1001, EXPOSE 8080, JAVA_TOOL_OPTIONS, HEALTHCHECK
ENTRYPOINT ["java", "-jar", "library.jar"]
```

```
# app/src/main/resources/application-prod.yml
server:
  port: 8080
  tomcat:
    threads:
      max: 100
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 10
  jpa:
    hibernate:
      ddl-auto: validate
  flyway:
    enabled: true
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
  endpoint:
    health:
      probes:
        enabled: true
# TODO: logging level, spring.session (Task 3), management.server.port (Task 5)
```

```
@startuml 02-artifacts
component "Catalog" as catalog
component "Loans" as loans
component "Members" as members
component "Notifications" as notif
component "Persistence" as persist
artifact "library-1.0.jar" as jar
artifact "itc/library:1.0\n(OCI image)" as image
artifact "V1__schema.sql .. V4" as sql
artifact "application-prod.yml" <<deployment spec>> as spec
node "lib-vm-01" <<device>> as vm
```

```
database "PostgreSQL 17" as db
jar ..> catalog : <<manifest>>
' TODO: manifest arrows for loans, members, notif, persist
image ..> jar : <<contains>>
sql ..> persist : <<manifest>>
' TODO: image ..> vm : <<deploy>>  and  sql ..> db : <<deploy>>
spec ..> jar
@enduml
```

EXPECTED OUTPUT

```
$ ./mvnw -q -DskipTests package && ls -lh target/*.jar
-rw-r--r-- 1 you you 48M  target/library-1.0.jar

$ docker build -f docker/Dockerfile -t itc/library:1.0 app/
 => [build 6/6] RUN ./mvnw -q -DskipTests package
 => [stage-1 4/4] COPY --from=build /src/target/library-1.0.jar library.jar
 => naming to docker.io/itc/library:1.0

$ docker run --rm -p 8080:8080 -e SPRING_PROFILES_ACTIVE=dev itc/library:1.0
...  The following 1 profile is active: "dev"
...  Tomcat started on port 8080 (http)
$ curl -s localhost:8080/actuator/health
{"status":"UP","groups":["liveness","readiness"]}
$ docker exec $(docker ps -q -f ancestor=itc/library:1.0) id
uid=1001(app) gid=1001(app)
```

Show hints

- The build context is `app/`, so the Dockerfile path needs `-f docker/Dockerfile`; `COPY` paths are relative to the context.
- `useradd --system --uid 1001 --create-home app` works on the Ubuntu-based Temurin image; `curl` is present there for the `HEALTHCHECK`.
- Spring Boot maps `DB_URL` to the placeholder `${DB_URL}` directly; relaxed binding also maps `SPRING_DATASOURCE_URL` to `spring.datasource.url`.
- In PlantUML a dashed arrow with a stereotype label is `a ..> b : <<manifest>>`. Direction: from the artifact to the component, from the artifact to the node.
- A digest is obtained after a push, or locally with `docker image inspect --format '{{.Id}}'` (image id) if you have no registry yet.

ACCEPTANCE CHECKLIST

- [ ] Multi-stage Dockerfile builds; the container runs as uid 1001 on a JRE 25 image with `MaxRAMPercentage` and a health check.

- [ ] `application-prod.yml` contains no secret; every required variable is listed in `.env.example` and the README table.
- [ ] The manifestation diagram links all five Lab-11 components to artifacts and all artifacts to nodes with «manifest» / «deploy».
- [ ] README states the unit of promotion and the image digest.

3 Production topology: zones, load balancer, replicas, broker

Medium **75 min**

GOAL

Design the production deployment for 5 000 members: a load balancer in front of two application instances, PostgreSQL primary and streaming replica, Redis as session store, RabbitMQ for overdue-loan notifications, and two external systems (the university identity provider for login, the payment provider for fines). Place everything in network zones with firewalls, then make the same topology run locally with Docker Compose.

STEPS

1. Draw `diagrams/03-prod-topology.puml` with three `rectangle` zones: **DMZ** (`lb-01` nginx with TLS termination, bastion host), **App zone** (`app-01`, `app-02` each with JVM 25 + `library-1.0.jar`, plus `worker-01` running `library-notifier.jar`), **Data zone** (`db-primary`, `db-replica`, `redis-01`, `mq-01`). Put the client devices and the two `cloud` externals (`University IdP (OIDC)`, `Payment provider`) outside the zones.
2. Label every path with protocol and port: HTTPS :443, HTTP :8080, JDBC :5432 (ssl), streaming replication :5432, RESP :6379, AMQP :5672, OIDC/HTTPS :443 (egress), payment HTTPS :443 (egress), SSH :22 via bastion only. Mark firewalls `FW-1`, `FW-2`, `FW-3` as named notes or barrier nodes.
3. Write `README.md#firewall-rules`: one table row per allowed flow (source zone, destination, port). Everything not listed is denied; state explicitly that the data zone accepts nothing from the DMZ.
4. Make the app horizontally scalable: add `spring-session-data-redis` and `spring-boot-starter-amqp` to the project; publish a `LoanOverdue` event from `Loans` and consume it in the notifier (`@RabbitListener`) which sends the e-mail. Sessions must survive a restart of one instance.
5. Write `docker/docker-compose.yml` reproducing the topology: `nginx` (upstream to `app1` and `app2`), `app1`, `app2` (same image, profile `prod`), `notifier`, `db` (postgres:17 with `healthcheck`), `db-replica` (optional: a second postgres with `pg_basebackup` start script, or

document why you skip it locally), `redis:7-alpine`, `rabbitmq:4-management`, and a `mailhog` or `mailpit` container standing in for the e-mail system. Use two Compose networks `frontend` and `backend` so nginx cannot reach the database.

6. Write `docker/nginx/library.conf` with `upstream library_app { least_conn; server app1:8080; server app2:8080; }`, a self-signed certificate for `localhost`, `proxy_set_header X-Forwarded-Proto https` and `location /actuator/ { deny all; }`.
7. Run `docker compose up -d`, log in through `https://localhost`, then `docker compose stop app1` and prove that the session and the next request still work on `app2`. Trigger an overdue notification and show the e-mail in Mailpit. Paste both transcripts into the README.

STARTER CODE

```
# docker/docker-compose.yml (skeleton)
services:
  nginx:
    image: nginx:1.27-alpine
    ports: ["443:443"]
    volumes:
      - ./nginx/library.conf:/etc/nginx/conf.d/default.conf:ro
      - ./nginx/certs:/etc/nginx/certs:ro
    networks: [frontend]
    depends_on: [app1, app2]
  app1: &app
    image: itc/library:1.0
    environment:
      SPRING_PROFILES_ACTIVE: prod
      DB_URL: jdbc:postgresql://db:5432/library
      DB_USER: library
      DB_PASSWORD: ${DB_PASSWORD}
      SPRING_DATA_REDIS_HOST: redis
      SPRING_RABBITMQ_HOST: rabbitmq
      SERVER_FORWARD_HEADERS_STRATEGY: framework
    networks: [frontend, backend]
    depends_on:
      db: { condition: service_healthy }
      redis: { condition: service_started }
      rabbitmq: { condition: service_healthy }
  app2:
    <<: *app
  notifier:
    image: itc/library-notifier:1.0
    # TODO: environment (SPRING_RABBITMQ_HOST, SPRING_MAIL_HOST=mailpit)
    networks: [backend]
  db:
```

```
    image: postgres:17
    environment:
      POSTGRES_DB: library
      POSTGRES_USER: library
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U library"]
      interval: 5s
      retries: 10
    networks: [backend]
  redis:
    image: redis:7-alpine
    networks: [backend]
  rabbitmq:
    image: rabbitmq:4-management
    ports: ["15672:15672"]
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 10s
    networks: [backend]
  mailpit:
    image: axllent/mailpit
    ports: ["8025:8025"]
    networks: [backend]
networks:
  frontend: {}
  backend: {}
volumes:
  pgdata: {}
```

```
// app: publish an event when a loan becomes overdue
@Service
public class OverdueLoanPublisher {
    private final RabbitTemplate rabbit;
    public OverdueLoanPublisher(RabbitTemplate rabbit) { this.rabbit = rabbit; }

    public void publish(LoanOverdue event) {
        rabbit.convertAndSend("library.events", "loan.overdue", event);
    }
}

// notifier: consume it (separate jar, separate node)
@Component
public class OverdueMailer {
    private final JavaMailSender mail;
    public OverdueMailer(JavaMailSender mail) { this.mail = mail; }
```

```
    @RabbitListener(queues = "loan-overdue-mail")
    public void onOverdue(LoanOverdue event) {
        // TODO: build and send the reminder e-mail
    }
}
```

EXPECTED OUTPUT

```
$ docker compose up -d && docker compose ps --format 'table
{{.Service}}\t{{.Status}}'
SERVICE    STATUS
app1       Up 40 seconds (healthy)
app2       Up 40 seconds (healthy)
db         Up 50 seconds (healthy)
mailpit    Up 50 seconds
nginx      Up 38 seconds
notifier   Up 39 seconds
rabbitmq   Up 50 seconds (healthy)
redis      Up 50 seconds

$ curl -k -c jar.txt -d 'username=alice&password=secret' https://localhost/login
$ curl -k -b jar.txt https://localhost/api/loans/mine | head -c 80
[{"id":17,"isbn":"9780134685991","dueDate":"2026-10-02"...
$ docker compose stop app1
$ curl -k -b jar.txt https://localhost/api/loans/mine | head -c 80
[{"id":17,"isbn":"9780134685991","dueDate":"2026-10-02"...   <- app2, same
session

$ docker compose logs notifier | tail -1
... OverdueMailer : reminder sent to alice@itc.edu for loan 17  (see
http://localhost:8025)
```

Show hints

- PlantUML: `rectangle "DMZ" { ... }` draws a zone; `cloud "University IdP" as idp` for externals; a firewall can be `node "FW-2" <<firewall>>` or simply a note on the crossing edge.
- YAML anchors (`&app` / `<<: *app`) keep app1 and app2 identical; only the service name differs.
- Redis-backed sessions need nothing beyond the starter and `@EnableRedisHttpSession` (or the auto-configuration in Boot 4); check `redis-cli keys 'spring:session:*'`.
- A self-signed cert: `openssl req -x509 -newkey rsa:2048 -nodes -keyout localhost.key -out localhost.crt -subj /CN=localhost -days 365`.
- If the replica is too much for a laptop, draw it in the diagram anyway and state in the README that Compose runs a single database.

ACCEPTANCE CHECKLIST

- [ ] The diagram shows three zones, three firewalls, LB, two app nodes, worker, primary + replica, Redis, RabbitMQ and two externals; every path has protocol and port.
- [ ] The firewall table lists only allowed flows and the data zone has no rule from the DMZ.
- [ ] Compose starts the whole topology with two networks; nginx cannot reach the database.
- [ ] Stopping app1 keeps the logged-in session working; an overdue notification arrives in Mailpit through RabbitMQ.

4

Containers and cloud: Kubernetes manifests and the cluster diagram

Medium **80 min**

GOAL

Deploy the library image to a local Kubernetes cluster with production-grade manifests, draw the cluster as a deployment diagram (nodes, pods, containers, Service, Ingress) and demonstrate what happens when one pod dies.

STEPS

1. Create a cluster with two worker nodes: `kind create cluster --config k8s/kind.yaml` (one control-plane, two workers; map ports 80/443 to the host). Install ingress-nginx and load the image: `kind load docker-image itc/library:1.0`.
2. Write `k8s/base/deployment.yaml`: `replicas: 2`, `envFrom` a ConfigMap and a Secret, `resources.requests` (500m CPU, 512Mi) and `limits` (1 CPU, 1Gi), `readinessProbe` on `/actuator/health/readiness` (period 5 s) and `livenessProbe` on `/actuator/health/liveness` (initial delay 30 s), `securityContext` with `runAsNonRoot`, and a `podAntiAffinity` rule that spreads pods over nodes.
3. Write `service.yaml` (ClusterIP 80 → 8080), `ingress.yaml` (host `library.local`, TLS secret `library-tls` from a self-signed cert, or cert-manager with a self-signed ClusterIssuer), `configmap.yaml` (`SPRING_PROFILES_ACTIVE`, `DB_URL`, Redis and RabbitMQ hosts) and `secret.yaml` (`DB_USER`, `DB_PASSWORD`; commit only a template, create the real one with `kubectl create secret`).
4. Add `hpa.yaml`: `minReplicas: 2`, `maxReplicas: 6`, CPU target 70 %. Deploy PostgreSQL, Redis and RabbitMQ in the cluster with simple manifests (a StatefulSet for PostgreSQL

with a PVC) or point the ConfigMap at the Compose containers on your host; state your choice in the README.

5. Apply everything with `kubectl apply -k k8s/base`, wait for `rollout status`, add `127.0.0.1 library.local` to your hosts file and open `https://library.local`. Show `kubectl get pods -o wide` proving the two pods are on different workers.
6. Draw `diagrams/04-kubernetes.puml`: cluster boundary, control plane node, two worker «device» nodes each with `containerd` as execution environment, pods as nested nodes with the `itc/library:1.0` container, the Service and Ingress as named paths, the data pods, the HPA range as multiplicity `[2..6]`.
7. Failover demo: run a loop `while true; do curl -sk https://library.local/actuator/health; sleep 0.5; done` in one terminal; in another `kubectl delete pod <one library pod>`. Record how many requests failed (expected: 0 or 1), how long the replacement pod took to become Ready, and explain in `README.md#failover` the sequence: endpoint removed from the Service → ReplicaSet creates a pod → readiness probe passes → endpoint added.

STARTER CODE

```
# k8s/base/deployment.yaml (fill the TODOs)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: library
  labels: { app: library }
spec:
  replicas: 2
  selector: { matchLabels: { app: library } }
  strategy:
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  template:
    metadata: { labels: { app: library } }
    spec:
      securityContext: { runAsNonRoot: true, runAsUser: 1001 }
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            podAffinityTerm:
              labelSelector: { matchLabels: { app: library } }
              topologyKey: kubernetes.io/hostname
      containers:
      - name: library
        image: itc/library:1.0
        imagePullPolicy: IfNotPresent
```

```
          ports: [{ containerPort: 8080, name: http }]
          envFrom:
          - configMapRef: { name: library-config }
          - secretRef:    { name: library-secrets }
          resources:
            requests: { cpu: 500m, memory: 512Mi }
            limits:   { cpu: "1",  memory: 1Gi }
          # TODO: readinessProbe (/actuator/health/readiness, periodSeconds 5)
          # TODO: livenessProbe  (/actuator/health/liveness, initialDelaySeconds
30)
---
# k8s/base/kustomization.yaml
resources:
- configmap.yaml
- deployment.yaml
- service.yaml
- ingress.yaml
- hpa.yaml


# k8s/kind.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  kubeadmConfigPatches:
  - |
    kind: InitConfiguration
    nodeRegistration:
      kubeletExtraArgs:
        node-labels: "ingress-ready=true"
  extraPortMappings:
  - { containerPort: 80,  hostPort: 80 }
  - { containerPort: 443, hostPort: 443 }
- role: worker
- role: worker
```

EXPECTED OUTPUT

```
$ kubectl apply -k k8s/base && kubectl rollout status deployment/library
deployment "library" successfully rolled out
$ kubectl get pods -l app=library -o wide
NAME                       READY   STATUS    NODE
library-6d8f9c7b5-2xk4q    1/1     Running   kind-worker
library-6d8f9c7b5-9pltz    1/1     Running   kind-worker2
$ kubectl get hpa library
NAME      REFERENCE            TARGETS   MINPODS   MAXPODS   REPLICAS
```

```
library   Deployment/library   12%/70%   2         6         2

# failover demo (second terminal)
$ kubectl delete pod library-6d8f9c7b5-2xk4q
pod "library-6d8f9c7b5-2xk4q" deleted
$ kubectl get pods -l app=library -w
library-6d8f9c7b5-2xk4q   1/1   Terminating
library-6d8f9c7b5-vq7d3   0/1   ContainerCreating
library-6d8f9c7b5-vq7d3   0/1   Running          <- not Ready yet: no traffic
library-6d8f9c7b5-vq7d3   1/1   Running          <- readiness passed after ~18 s
# curl loop: 0 failed requests, all served by the surviving pod
```

Show hints

- Install ingress-nginx for kind with the manifest from the ingress-nginx project (`deploy/static/provider/kind/deploy.yaml`) and wait for the controller pod.
- The HPA needs metrics-server; on kind install it with `--kubelet-insecure-tls`. Without it the HPA shows `<unknown>`, which is acceptable if you say so.
- `kubectl create secret generic library-secrets --from-literal=DB_USER=library --from-literal=DB_PASSWORD=...`; commit `secret.example.yaml` only.
- If pods restart in a loop, the liveness probe fires before Spring Boot finished starting: raise `initialDelaySeconds` or use a `startupProbe`.
- For the PlantUML cluster diagram, a pod is just a nested `node "pod library-…"` inside the worker node with `<<container>>` artifacts inside it.

ACCEPTANCE CHECKLIST

- [ ] Deployment has both probes, resource requests and limits, non-root security context and anti-affinity; Service, Ingress with TLS, ConfigMap, Secret template and HPA exist and apply cleanly.
- [ ] `https://library.local` serves the app; the two pods run on different worker nodes.
- [ ] The cluster diagram shows nodes, containerd, pods, containers, Service, Ingress, data pods and the `[2..6]` multiplicity.
- [ ] The failover demo is documented with measured numbers and the four-step explanation.

5 Environments, security boundaries and monitoring Hard

90 min

GOAL

Show that one artifact serves dev, test and prod with different deployment specifications; make the security boundaries explicit (TLS termination, secrets path, management port); add the monitoring stack (Prometheus, Grafana, centralised logs) to the diagram and to the cluster; and write two runbooks that an on-call engineer could follow using only your diagrams.

STEPS

1. Write the environment table in `README.md#environments` (rows: profile, nodes, database, schema strategy, secrets source, TLS, replicas, logging, data). Create `k8s/overlays/dev`, `test`, `prod` Kustomize overlays that patch only replicas, the ConfigMap and the Ingress host; the image reference stays in `base`.
2. Draw three variants `diagrams/05-env-dev.puml`, `05-env-test.puml`, `05-env-prod.puml`. Each shows the same artifact `itc/library@sha256:…` (same digest written on all three) with a different deployment specification box: dev = laptop, H2, profile `dev`; test = single VM with Compose, profile `test`; prod = cluster, managed PostgreSQL primary + standby, profile `prod`, replicas `[2..6]`.
3. Security boundaries in the prod diagram: draw the trust boundary as a dashed region; mark where TLS terminates (Ingress) and that traffic inside is HTTP :8080 (or mTLS if you install a mesh); move Actuator to `management.server.port: 8081`; add a NetworkPolicy that allows :8080 only from the ingress namespace and :8081 only from Prometheus; draw the secret path Vault / Kubernetes Secret → pod as a red dashed arrow.
4. Monitoring: install kube-prometheus-stack (or plain Prometheus + Grafana manifests) and Loki + Promtail (or a simpler `kubectl logs`-based sink if resources are tight). Configure a scrape job for `/actuator/prometheus` on port 8081 via pod labels; switch the app to JSON logs (`logging.structured.format.console: ecs`). Import a Grafana dashboard showing `http_server_requests_seconds` p95 and `hikaricp_connections_active`.
5. Add the monitoring flows to the prod diagram as blue dashed lines: Prometheus → pods (pull :8081), pods → Promtail → Loki (push), Grafana → on-call (alert). Define two alert rules: p95 latency > 500 ms for 5 min, and available replicas < 2.
6. Write `runbooks/database-failover.md`: symptoms (readiness DOWN, `hikaricp_connections_pending` rising), detection (alert), decision (promote standby or wait), steps with exact commands, verification (readiness UP, writes succeed), rollback. Reference the diagram elements by name. Write `runbooks/rolling-update.md` the same way for shipping `itc/library:1.1` by digest with `maxUnavailable: 0`, including how to watch the rollout and how to `rollout undo`.
7. Exercise both runbooks once on your cluster (simulate the DB failure by scaling the PostgreSQL StatefulSet to 0 for a minute) and paste the observed timeline (alert fired, readiness DOWN, recovery) into the runbooks.

STARTER CODE

```
# k8s/overlays/prod/kustomization.yaml
resources: [../../base]
namespace: library-prod
patches:
- patch: |-
    - op: replace
      path: /spec/replicas
      value: 3
  target: { kind: Deployment, name: library }
configMapGenerator:
- name: library-config
  behavior: merge
  literals:
  - SPRING_PROFILES_ACTIVE=prod
  - MANAGEMENT_SERVER_PORT=8081
  - DB_URL=jdbc:postgresql://db-primary.library-prod.svc:5432/library
---
# k8s/base/networkpolicy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: library }
spec:
  podSelector: { matchLabels: { app: library } }
  policyTypes: [Ingress]
  ingress:
  - from:
    - namespaceSelector:
        matchLabels: { kubernetes.io/metadata.name: ingress-nginx }
    ports: [{ port: 8080 }]
  - from:
    - namespaceSelector:
        matchLabels: { kubernetes.io/metadata.name: monitoring }
    ports: [{ port: 8081 }]
---
# monitoring: ServiceMonitor for kube-prometheus-stack
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata: { name: library, labels: { release: kube-prometheus-stack } }
spec:
  selector: { matchLabels: { app: library } }
  endpoints:
  - port: management
    path: /actuator/prometheus
    interval: 15s
```

```
# runbooks/database-failover.md (skeleton)
# Runbook: database failover

## Symptoms
- Alert "LibraryReadinessDown" (diagram: Grafana -> on-call)
- /actuator/health/readiness on library pods returns DOWN, component db
- hikaricp_connections_pending > 0

## Decision
- Primary unreachable > 60 s and standby lag < 30 s  -> promote standby
- otherwise wait / escalate to the DBA

## Steps
1. TODO exact commands (promote, repoint db-primary Service or DNS)
2. TODO verify: readiness UP on all pods, one write succeeds

## Rollback
- TODO

## Observed timeline (exercise on YYYY-MM-DD)
- 10:02:10 primary scaled to 0
- 10:02:25 readiness DOWN on 2/2 pods, Ingress returns 503
- ...
```

EXPECTED OUTPUT

```
$ kubectl apply -k k8s/overlays/prod && kubectl -n library-prod get deploy,hpa
NAME                      READY   UP-TO-DATE   AVAILABLE
deployment.apps/library   3/3     3            3
NAME      REFERENCE            TARGETS   MINPODS   MAXPODS   REPLICAS
library   Deployment/library   9%/70%    2         6         3

$ kubectl -n library-prod exec deploy/library -- \n    curl -s
localhost:8081/actuator/prometheus | grep -c '^http_server'
14
$ kubectl -n library-prod exec deploy/library -- \n    curl -s -o /dev/null -w
'%{http_code}' localhost:8080/actuator/health
404                                              <- Actuator no longer on the
app port

# during the failover exercise
$ kubectl -n library-prod get pods -l app=library
library-7c9d5b4f6-8h2kd   0/1   Running     <- readiness DOWN, liveness still
OK, no restarts
library-7c9d5b4f6-mq4vz   0/1   Running
```

```
library-7c9d5b4f6-tt8xn   0/1   Running
# Grafana: alert LibraryReadinessDown FIRING at 10:02:40, RESOLVED at 10:04:05
```

Show hints

- When the management port changes, the probes in the Deployment must point at 8081 too; expose it as a named container port `management` so the ServiceMonitor can reference it.
- Kustomize patch precedence: overlays patch the base; keep the image digest in `base/kustomization.yaml` under `images:` so all three overlays share it.
- Prometheus alert rule example: `histogram_quantile(0.95, sum(rate(http_server_requests_seconds_bucket[5m])) by (le)) > 0.5`.
- Readiness DOWN with zero restarts is the proof that liveness and readiness are separated correctly; a restart count going up means the liveness probe includes the database.
- A runbook is judged by whether a stranger can execute it: every step names a command, a diagram element and an expected result.

ACCEPTANCE CHECKLIST

- [ ] Environment table and three diagram variants share one image digest and differ only in the deployment specification; overlays apply for dev, test and prod.
- [ ] Prod diagram shows the trust boundary, TLS termination point, secret injection path, management port 8081 and a NetworkPolicy that enforces it.
- [ ] Prometheus scrapes the app, Grafana shows the two panels, logs are JSON and centralised, two alert rules exist; the flows are drawn as dashed telemetry lines.
- [ ] Both runbooks contain symptoms, decision, exact steps, verification, rollback and an observed timeline from a real exercise.

★

Challenge: blue/green release, pipeline, capacity and disaster recovery

Hard **120 min · optional, extra credit**

SCENARIO

Semester start is in two weeks. The library board wants zero-downtime releases, proof that the platform can take the first-day rush (every student borrows textbooks in the same morning), and a written plan for losing the whole database. You answer with diagrams and numbers, not adjectives.

REQUIREMENTS

1. **Blue/green or canary design.** Choose one. Draw three deployment diagrams `diagrams/C1-before.puml`, `C1-during.puml`, `C1-after.puml`: blue (`library:1.0`) and green (`library:1.1`) Deployments side by side, the Service or Ingress weight that moves traffic (blue/green: switch the selector; canary: an Ingress with `canary-weight: 10`), the shared database, and what happens to Redis sessions and to in-flight RabbitMQ messages during the switch. Write the switch and the rollback as `kubectl` commands.
2. **Pipeline.** Write `.github/workflows/deploy.yml` with jobs `test` (runs the Chapter 06 JUnit 6 / Mockito 5 suite with Testcontainers PostgreSQL), `build-image` (multi-stage build, push to `ghcr.io`, output the digest), `deploy-test` (Compose on the test VM or the test overlay), `deploy-prod` (environment with manual approval, applies the prod overlay with the digest, waits for `rollout status`, runs a smoke test, and performs the blue/green switch from step 1). Draw the pipeline as a deployment diagram of the CI runner, registry and cluster.
3. **Capacity estimate.** Assume 5 000 members, 40 % active on day one between 08:00 and 12:00, 25 requests per session, p95 service time 80 ms per request on one pod at 1 CPU. Compute: average and peak (×4) requests per second, pods needed at 70 % target utilisation, Tomcat thread count per pod, Hikari pool size per pod, total PostgreSQL connections, and the HPA min/max. Show every step of the arithmetic in `README.md#capacity` and put the results as multiplicities and deployment specs on the prod diagram.
4. **Disaster recovery.** Define RPO and RTO for the library (propose 15 min / 1 h and justify). Draw `diagrams/C4-backup.puml`: nightly base backup + continuous WAL archiving to object storage in another region, the restore path to a fresh PostgreSQL node, and the DNS or Service switch. Write `runbooks/disaster-recovery.md` with the restore commands and do one real restore into a scratch namespace, recording the measured RTO.

SKELETON

```
# .github/workflows/deploy.yml (skeleton)
name: deploy
on:
  push:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with: { distribution: temurin, java-version: "25", cache: maven }
    - run: ./mvnw -B verify          # JUnit 6 + Mockito 5 + Testcontainers
      working-directory: app
```

```
  build-image:
    needs: test
    runs-on: ubuntu-latest
    permissions: { packages: write, contents: read }
    outputs:
      digest: ${{ steps.push.outputs.digest }}
    steps:
    - uses: actions/checkout@v4
    - uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}
    - id: push
      uses: docker/build-push-action@v6
      with:
        context: app
        file: docker/Dockerfile
        push: true
        tags: ghcr.io/${{ github.repository }}/library:${{ github.sha }}
  deploy-test:
    needs: build-image
    # TODO: apply overlays/test with the digest, run smoke test
  deploy-prod:
    needs: [build-image, deploy-test]
    environment: production        # manual approval in repo settings
    # TODO: apply overlays/prod as "green", verify, switch Service selector
```

```
# README.md#capacity (fill in every line)
active members on day one      = 5 000 × 40 %                 = 2 000
requests in the 4-hour window  = 2 000 × 25                   = 50 000
average req/s                  = 50 000 / (4 × 3 600)         = 3.5
peak req/s (×4)                = 14
capacity of one pod at 1 CPU   = 1 / 0.080 s ≈ 12.5 req/s (100 %)
pods at 70 % target            = 14 / (12.5 × 0.7)            = 1.6 -> 2 (min 2
for HA)
Tomcat threads per pod         = TODO (concurrency = req/s × service time, plus
headroom)
Hikari pool per pod            = TODO
PostgreSQL max_connections     = pods_max × pool + 10 for admin = TODO
HPA                            = min 2, max TODO
```

EXPECTED OUTPUT

```
$ kubectl -n library-prod get deploy
NAME           READY   UP-TO-DATE   AVAILABLE
```

```
library-blue    3/3     3           3
library-green   3/3     3           3
$ kubectl -n library-prod patch service library \n    -p '{"spec":{"selector":
{"app":"library","color":"green"}}}'
service/library patched
# curl loop during the switch: 0 failed requests; sessions preserved (Redis)

$ gh run view --job deploy-prod
✓ test          2m 41s
✓ build-image   1m 58s   digest=sha256:3f1c…
✓ deploy-test   1m 05s
✓ deploy-prod   2m 12s   (approved by: board-ops)

# disaster-recovery exercise
restore started 14:10:00, WAL replayed to 14:02:31 (RPO 7 min 29 s)
service UP 14:41:15 (RTO 31 min)
```

Show hints

- Blue/green with one Service: both Deployments carry `app: library` plus `color: blue|green`; the Service selector includes `color`. Switching is one `kubectl patch`; rollback is the same patch with the old colour.
- Database schema changes must be backward compatible for the duration of the overlap (expand/contract): mention it on the "during" diagram.
- `docker/build-push-action` exposes `steps.<id>.outputs.digest`; deploy with `image@digest`, never with a tag.
- Tomcat threads: concurrent requests ≈ req/s × service time; 14 × 0.08 ≈ 1.1 in flight on average, so the default 200 is far more than needed; explain why you still keep headroom for slow requests.
- For the DR exercise, `pg_basebackup` plus `archive_command` to a MinIO bucket is enough locally; the restore is `restore_command` plus a `recovery_target_time`.

ACCEPTANCE CHECKLIST

- [ ] Before/during/after diagrams and the switch + rollback commands exist; the session and message questions are answered on the diagram.
- [ ] The pipeline runs the Chapter 06 tests, pushes by digest, deploys test then prod with approval, and is drawn as a deployment diagram.
- [ ] The capacity estimate shows all arithmetic and its results appear as multiplicities and deployment specs on the prod diagram.
- [ ] RPO/RTO are defined and justified, the backup diagram exists, and one real restore was measured and documented.

Grading rubric

Component	Points	Full marks when…
Task 1 · First deployment diagram	10	Correct UML 2.5.1 notation: stereotyped 3D nodes, nesting, artifacts with icon, labelled paths, deployment spec; open checklist points identified.
Task 2 · Artifacts and specifications	15	Multi-stage non-root image builds and runs; prod spec without secrets; manifestation diagram links all five components to artifacts and nodes.
Task 3 · Production topology	20	Three zones with firewalls and complete labelled paths; firewall table; Compose topology with two networks; session survives an instance stop; notification flows through RabbitMQ.
Task 4 · Containers and cloud	20	Complete manifests with probes, limits, HPA, TLS Ingress; pods spread across nodes; cluster diagram; measured failover with explanation.
Task 5 · Environments, security, monitoring	25	Three environment variants with one digest; trust boundary, TLS, secrets and management port enforced; monitoring stack and alerts wired and drawn; two exercised runbooks.
Diagram / code quality	10	Consistent naming across all diagrams and manifests, readable layout, PNG exports match sources, README navigable, no secret in git.
★ Challenge (extra credit)	+20	Blue/green or canary with rollback, digest-based pipeline with approval, capacity arithmetic reflected on the diagram, measured DR restore.
Total	**100 (+20)**	

Automatic deductions: −5 per communication path without protocol and port (max −15) · −5 for a class or component drawn directly on a node instead of an artifact · −5 for a JVM, Tomcat, Docker or PostgreSQL stereotyped «device», or a VM stereotyped «executionEnvironment» · −10 for any credential committed in a manifest, Compose file, `application-*.yml` or diagram · −5 for a liveness probe that includes the database · −5 for a container running as root or a fixed `-Xmx` in a container with a memory limit · −5 for a diagram delivered as PNG only (no source text).

Submission

1. Repository layout:

```
library-deploy/
├── README.md                    # firewall rules, environments, failover,
capacity
├── diagrams/
│   ├── 01-single-server.puml / .png
│   ├── 02-artifacts.puml / .png
│   ├── 03-prod-topology.puml / .png
│   ├── 04-kubernetes.puml / .png
│   ├── 05-env-dev.puml  05-env-test.puml  05-env-prod.puml  (+ .png)
│   └── C1-before/during/after.puml, C2-pipeline.puml, C4-backup.puml
(challenge)
├── app/                         # Spring Boot project incl. application-
prod.yml
├── docker/
│   ├── Dockerfile
│   ├── docker-compose.yml  .env.example
│   ├── nginx/library.conf  nginx/certs/
│   └── prometheus/prometheus.yml
├── k8s/
│   ├── kind.yaml
│   ├── base/       # deployment, service, ingress, configmap,
secret.example,
│   │               # hpa, networkpolicy, kustomization
│   └── overlays/dev  overlays/test  overlays/prod
├── runbooks/
│   ├── database-failover.md  rolling-update.md
│   └── disaster-recovery.md  (challenge)
└── .github/workflows/deploy.yml  (challenge)
```

2. Verification commands that must pass from a clean checkout:

```
java -jar plantuml.jar -tpng -o . diagrams/*.puml      # every diagram
renders
docker build -f docker/Dockerfile -t itc/library:1.0 app/
docker compose -f docker/docker-compose.yml config -q   # Compose file valid
kubectl kustomize k8s/overlays/prod | kubectl apply --dry-run=client -f -
git grep -iE 'password[:=] *[^$<{ ]' -- ':!*.example*' \n  && echo "SECRET
FOUND" || echo "no secret in git"
```

3. Branch `lab-12`, merge request titled `Lab-12 – <your name>`. The MR description links each acceptance checklist item to the file or diagram that proves it.

Back to Chapter 12 slides | Retake the chapter quiz | ← Lab-11